

BMIT3173 Integrative Programming

ASSIGNMENT 202605

Student Name : Wong Men Jing

Student ID : 25WMR09788

Programme : Bachelor in Information Technology (Honours) (Information Security)

Tutorial Group : 4

System Title : NutriShare: Surplus Food Redistribution Platform

Chosen SDG : SDG 2: Zero Hunger

Modules : Module 4 — Inventory & Food Safety Compliance

AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

Mandatory submission: Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

AI Usage Disclosure Form

Declaration (tick one):

☐ No AI tools were used in the preparation of this report.

☒ AI tools were used as declared in the table below.

AI Tool Used (name & version)	Purpose / How It Was Used	Report Section(s) Affected
Grammarly (2026 Edition)	Used solely for language polish, tone consistency, sentence phrasing, and spelling verification of my original written descriptions for food inventory safety and strategy pattern analysis without altering technical logic or analysis (Grammarly Inc., 2026).	Sections 1.1, 2.2, 4.1, 4.3, 5.1, 6.1

If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: *Wong*

Date: 06/09/2026

Table of Contents

1. Introduction to the System	4
1.1 System Overview	4
1.2 Chosen Sustainable Development Goal (SDG)	5
1.3 System Contribution to SDG 2 & Scope	6
2. Module Description	7
2.1 Scope of Module 4: Inventory & Food Safety Compliance	7
2.2 Functional Breakdown & Class Paths	8
3. Entity Classes	16
3.1 Entity Class Diagram	16
3.2 Entity Class Implementation (Eloquent ORM Mapping)	17
4. Design Pattern	26
4.1 Description of Design Pattern: Strategy Pattern (GoF Behavioural)	26
4.2 Implementation of Design Pattern	28
4.3 Justification of Design Pattern	38
5. Software Security	39
5.1 Potential Threats and Attacks	39
5.2 Secure Coding Practices & Implementation	40
6. Web Services	44
6.1 Web Service Exposure	44
6.2 Web Service Consumption	52
7. References	55
8. Appendices	56
Appendix A: Automated Testing Results	56
Appendix B: GitHub Repository URL	58

1. Introduction to the System

1.1 System Overview

NutriShare is an enterprise-grade web platform architected to combat urban food waste and address systemic food insecurity. Modern commercial food establishments including hypermarkets, bakery chains, catering enterprises, and hotels frequently generate substantial volumes of wholesome, edible surplus food due to strict cosmetic retail standards, inventory overstocking, and approaching "best before" thresholds (Food and Agriculture Organisation [FAO], 2023). Simultaneously, charitable food banks, soup kitchens, and non-governmental organisations (NGOs) struggle with unpredictable supply lines and logistically fragmented distribution channels.

NutriShare bridges this critical coordination gap through an integrated, role-based platform. Commercial food donors publish available surplus food inventory in real-time, verified NGOs claim appropriate lots based on community need, and logistics dispatchers coordinate physical collection via transparent, state-driven workflows. By replacing ad-hoc communication with a centralised, auditable digital infrastructure, NutriShare streamlines the entire surplus food redistribution pipeline from initial donation to final beneficiary handover (FAO, 2023).

1.2 Chosen Sustainable Development Goal (SDG)

The primary objective of NutriShare aligns with **United Nations Sustainable Development Goal 2: Zero Hunger**. This goal seeks to eliminate hunger, improve food security and nutrition, and support sustainable agriculture (United Nations, 2015). NutriShare contributes through the redistribution of surplus food and the provision of information that supports its handling, storage, and allocation.

Key SDG Targets Related to the System:

1. **Target 2.1: Access to Safe and Nutritious Food:** This target seeks to ensure that everyone, particularly people experiencing poverty or vulnerability, has reliable access to adequate and nutritious food. NutriShare supports this objective by connecting food donors with organisations that distribute food assistance. Module 4 contributes by making stored food, expiry dates, and allergen information visible to inventory managers (United Nations, 2015).
2. **Target 2.2: Addressing Malnutrition & Vulnerability:** This target addresses malnutrition and the nutritional needs of vulnerable population groups. NutriShare provides indirect support by helping organisations identify and redistribute available food across diverse nutritional categories (e.g., fresh produce, protein, dairy). However, the system does not assess individual dietary requirements or measure clinical improvements in nutritional health (United Nations, 2015).
3. **Related Target 12.3: Halving Food Loss and Waste:** Although SDG 2 is the primary goal, NutriShare directly reinforces SDG 12: Responsible Consumption and Production. Target 12.3 addresses reductions in retail and consumer food waste and losses throughout food supply chains (United Nations, 2015). Inventory visibility, thermal storage compliance, and expiry countdowns empower organisations to dispatch items before degradation occurs.

1.3 System Contribution to SDG 2 & Scope

NutriShare supports surplus food redistribution through the following functions, with Module 4 contributing primarily to inventory organisation and food safety compliance:

- **Target Users and Beneficiaries:** Direct users include commercial donors, NGO personnel, administrators, and moderators. Module 4 primarily serves NGO warehouse staff responsible for storage, pantry handling, and inventory allocation, while administrators provide governance. The intended beneficiaries include low-income households, individuals experiencing food insecurity, and community members receiving aid via soup kitchens and welfare shelters (FAO, 2023; United Nations, 2015).
- **Inventory Visibility and Storage Planning:** Organisations can register multiple storage facilities and inspect recorded capacity, occupancy, and item headcounts. Centralised inventory records help staff locate available food and assess storage availability, supporting logistics planning when shipments arrive.
- **Expiry and Allergen Awareness:** Each food record includes an expiry timestamp, thermal storage requirements, and standardised allergen tags. The inventory interface highlights expiring items with high-contrast warning badges. Physical inspection and appropriate food-handling procedures remain mandatory because recorded expiry data alone cannot substitute for sensory evaluation.
- **Information Sharing and Reporting:** Module 4 exposes RESTful web services returning inventory status and safety checks, facilitating cross-module automated inflows from claims and logistics. The module also provides RFC 4180-compliant CSV exports of facility summaries for governance and regulatory audits (Shafranovich, 2005).
- **Scope Boundaries:** Module 4 covers facility registration, item intake, capacity telemetry, allergen tracking, multi-channel notification dispatching, and audit reporting. It does not provide hardware IoT temperature monitoring, laboratory microbiology testing, or automated medical food certification.

2. Module Description

2.1 Scope of Module 4: Inventory & Food Safety Compliance

As the lead software engineer for **Module 4 (Inventory & Food Safety Compliance)**, I architected and implemented the multi-facility storage management system, granular food item pantry monitoring, dynamic allergen risk identification, real-time expiry countdown telemetry, RFC 4180-compliant CSV facility audit exporting, and runtime-swappable multi-channel notification dispatching powered by the **Strategy Pattern**. Furthermore, Module 4 enforces rigorous food safety compliance controls to guarantee that perishable items are stored under appropriate thermal conditions (dry, cold, frozen, ambient) and strictly vetted prior to humanitarian distribution.

2.2 Functional Breakdown & Class Paths

F4.1: Multi-Location Storage Facility Management & Leaflet Geolocation Pinning

- **Description:** Enables authenticated humanitarian NGOs, System Admins, and Platform Moderators to register, configure, and manage diverse food storage facilities (Dry Storage, Cold Rooms, Blast Freezers, and Ambient Pantries). Integrated with OpenStreetMap and the Leaflet.js interactive mapping engine, the module supports real-time reverse geocoding to accurately capture geographical store coordinates, street addresses, and maximum holding capacities in kilograms.
- **Class Paths:**
 - **Controller:** `app/Http/Controllers/InventoryController.php` (create, store)
 - **Form Request:** `app/Http/Requests/StoreInventoryLocationRequest.php`
 - **Blade View:** `resources/views/inventory/create.blade.php`

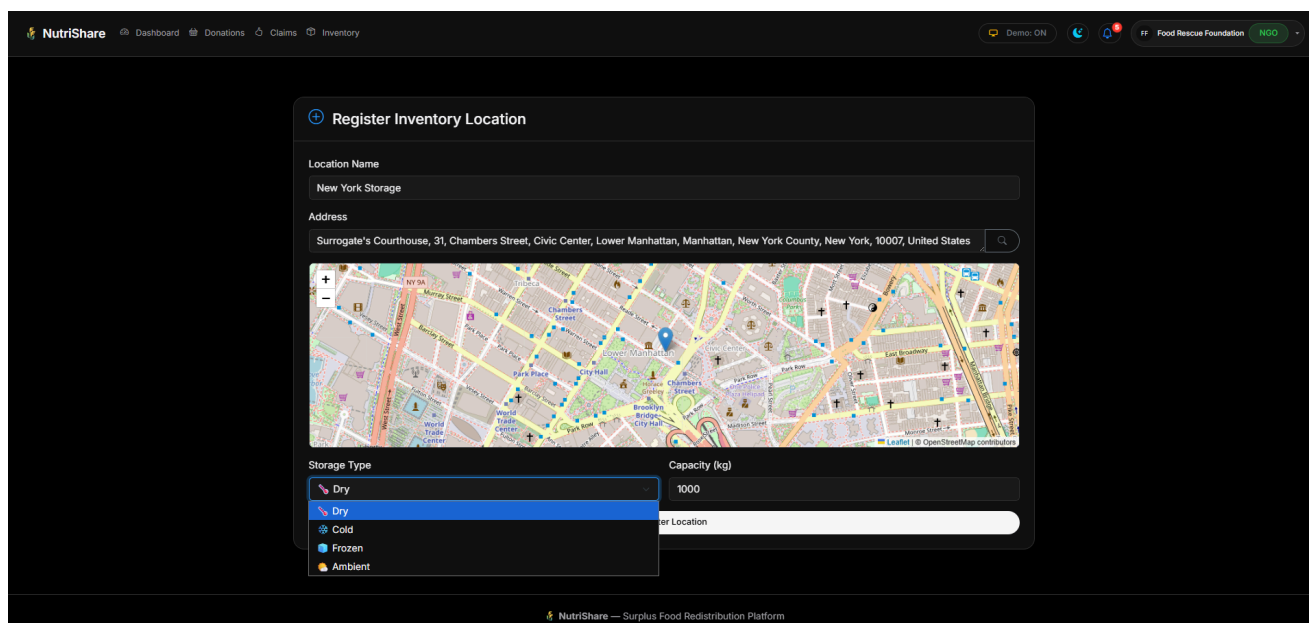


Figure 4.1: Multi-Location Storage Facility Registration Interface with Interactive Map Pinning

F4.2: Dual-Mode Inventory Facility Directory (Interactive Table & Visual Grid Views)

- **Description:** Provides a dual-mode visualisation dashboard displaying registered storage facilities across the ecosystem. Users can toggle seamlessly between an information-dense administrative table view and a responsive visual card grid view. Each facility card displays active occupancy versus maximum capacity with real-time percentage progress meters, total constituent item counters, and managing organisation tags.
- **Class Paths:**
 - **Controller:** `app/Http/Controllers/InventoryController.php` (index)
 - **Repository:** `app/Repositories/InventoryRepository.php`
 - **Blade View:** `resources/views/inventory/index.blade.php`

Location Name	Storage Type	Managing Organization	Capacity / Items	Action
Central Storage Facility 100 Community Way, KL	Dry Storage	Food Rescue Foundation	2 Items 828.0/2,000.0 kg	Manage Pantry
Cold Storage Blast Freezer 100 Community Way, Block B, KL	Frozen Storage	Food Rescue Foundation	2 Items 349.0/800.0 kg	Manage Pantry
Dry Pantry Storage Depot 100 Community Way, Room A, KL	Ambient Storage	Food Rescue Foundation	1 Items 367.0/1,200.0 kg	Manage Pantry
Cheras Emergency Relief Food Bank Cheras Community Center, KL	Ambient Storage	Food Rescue Foundation	1 Items 578.0/2,500.0 kg	Manage Pantry

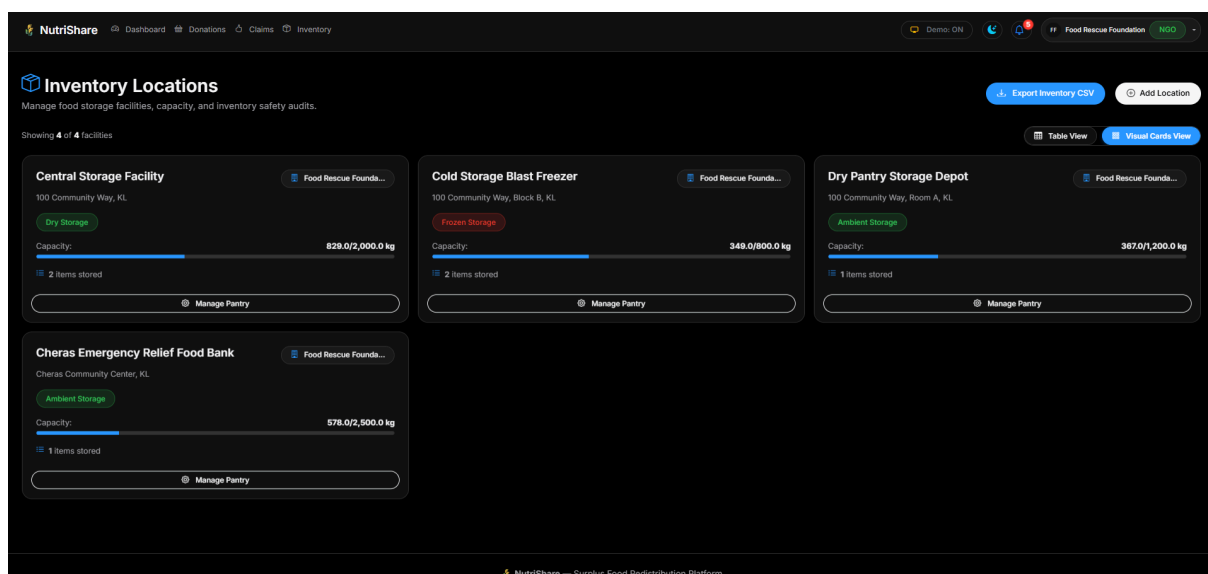
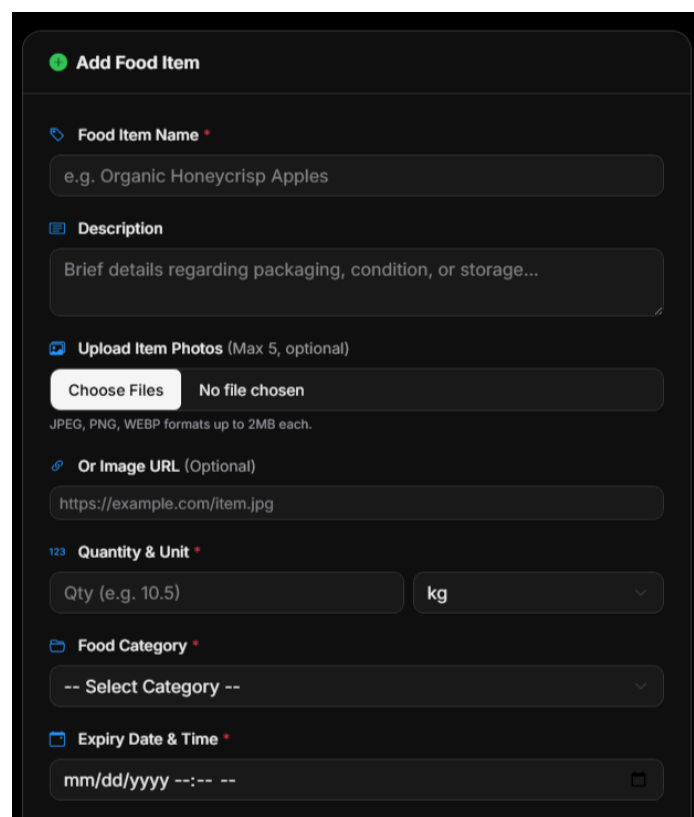


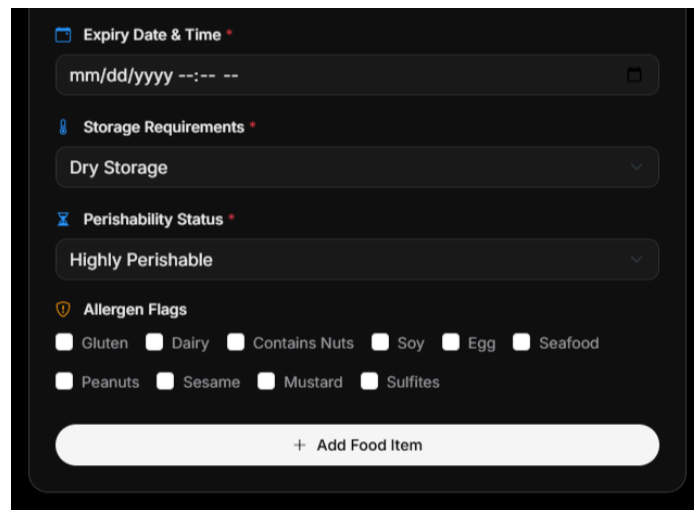
Figure 4.2: Filterable Storage Facilities Directory with Capacity Utilisation Progress Meters

F4.3: Perishable Food Item Ingestion & Multi-Media Photographic Documentation

- **Description:** Allows NGO pantry managers to log incoming recovered food supplies into specific warehouse locations. The intake form captures item nomenclature, categorical classifications, precise metrics (quantities and units), physical storage specifications, perishability classifications, and multi-image uploads (supporting up to 5 high-resolution photographs or direct external photo URLs) stored on public disk storage with interactive fullscreen modal inspection.
- **Class Paths:**
 - **Controller:** [app/Http/Controllers/InventoryController.php](#) (addFoodItem)
 - **Form Request:** [app/Http/Requests/StoreFoodItemRequest.php](#)
 - **Blade View:** [resources/views/inventory/show.blade.php](#)



The screenshot displays a web form titled "Add Food Item" with a green status indicator. The form is organized into several sections: 1. "Food Item Name" with a text input field containing the placeholder "e.g. Organic Honeycrisp Apples". 2. "Description" with a larger text area containing the placeholder "Brief details regarding packaging, condition, or storage...". 3. "Upload Item Photos (Max 5, optional)" featuring a file upload button labeled "Choose Files" and a status "No file chosen", with a note below stating "JPEG, PNG, WEBP formats up to 2MB each." 4. "Or Image URL (Optional)" with a text input field containing the placeholder "https://example.com/item.jpg". 5. "Quantity & Unit" with a numeric input field labeled "Qty (e.g. 10.5)" and a dropdown menu currently set to "kg". 6. "Food Category" with a dropdown menu showing "-- Select Category --". 7. "Expiry Date & Time" with a date and time picker showing the format "mm/dd/yyyy --:-- --".



The image shows a dark-themed user interface for a 'Perishable Food Item Intake Form'. It contains several sections: 'Expiry Date & Time' with a date-time input field showing 'mm/dd/yyyy --:-- --'; 'Storage Requirements' with a dropdown menu set to 'Dry Storage'; 'Perishability Status' with a dropdown menu set to 'Highly Perishable'; and 'Allergen Flags' with a grid of checkboxes for Gluten, Dairy, Contains Nuts, Soy, Egg, Seafood, Peanuts, Sesame, Mustard, and Sulfites. At the bottom is a large button labeled '+ Add Food Item'.

Expiry Date & Time *

mm/dd/yyyy --:-- --

Storage Requirements *

Dry Storage

Perishability Status *

Highly Perishable

Allergen Flags

☐ Gluten ☐ Dairy ☐ Contains Nuts ☐ Soy ☐ Egg ☐ Seafood

☐ Peanuts ☐ Sesame ☐ Mustard ☐ Sulfites

+ Add Food Item

Figure 4.3: Perishable Food Item Intake Form with Storage Condition and Media Selectors

F4.4: Allergen Tagging & Automated Expiry Threshold Compliance Tracking

- **Description:** Enforces strict food safety compliance by associating food items with critical allergen tags (Gluten, Dairy, Nuts, Soy, Egg, Seafood) via an Eloquent Many-to-Many relationship. The system evaluates item expiration timestamps against current system clocks in real time, rendering high-contrast safety warning badges (e.g., Red "Expired" tags) to avert accidental distribution of compromised food supplies to vulnerable communities.
- **Class Paths:**
 - **Controller:** [app/Http/Controllers/InventoryController.php](#) (show)
 - **Models:** [app/Models/FoodItem.php](#), [app/Models/AllergenTag.php](#)
 - **Blade View:** [resources/views/inventory/show.blade.php](#)

Item	Qty	Category	Allergens	Expires	Storage
Fresh Kale & Spinach Bunches	70.50 kg	Fresh Produce	Dairy	🕒 10 Sep 2026	Cold
Frozen Sweet Corn & Green Peas	60.00 kg	Frozen Foods	Sulfites	🕒 06 Dec 2026	Frozen

Figure 4.4: Food Pantry Inspection View with Prominent Allergen Flags and Expiry Indicators

F4.5: Storage Facility Capacity Audit & RFC 4180-Compliant CSV Exporter

- **Description:** Generates streamed RFC 4180-compliant CSV facility audit reports on demand (Shafranovich, 2005). The export streams facility identifiers, warehouse titles, physical addresses, thermal storage types, managing organisation names, total stored inventory items, active occupancy (kg), total capacity (kg), and storage utilisation percentages, equipped with UTF-8 Byte Order Marks (BOM) for seamless Microsoft Excel rendering.
- **Class Paths:**
 - **Controller:** [app/Http/Controllers/InventoryController.php \(exportCsv\)](#)

	A	B	C	D	E	F	G	H	I
1	ID	Facility Name	Address	Storage Type	Managing Organization	Stored Items Count	Occupancy (kg)	Capacity (kg)	Utilization %
2	1	Central Storage Facility	100 Community Way, KL	DRY	Food Rescue Foundation	2	829	2000	41.40%
3	2	Cold Storage Blast Freezer	100 Community Way, Block B, KL	FROZEN	Food Rescue Foundation	2	349	800	43.60%
4	3	Dry Pantry Storage Depot	100 Community Way, Room A, KL	AMBIENT	Food Rescue Foundation	1	367	1200	30.60%
5	10	Cheras Emergency Relief Food Bank	Cheras Community Center, KL	AMBIENT	Food Rescue Foundation	1	578	2500	23.10%

Figure 4.5: Generated Inventory Facilities CSV Audit Report Output

F4.6: Dynamic Notification Dispatching via Strategy Pattern

- **Description:** After a food item is added to an inventory facility, the system invokes a notification dispatcher to send a confirmation to the authenticated user. The Strategy Pattern selects the notification channel at runtime according to the user's notification preference (Gamma et al., 1994). **EmailStrategy** uses Laravel's mail service, while **SMSSStrategy** records a simulated SMS notification in the application log for demonstration purposes. This separates channel-specific delivery logic from the inventory controller.
- **Class Paths:**
 - **Strategy** **Contract:**
`app/Strategies/Notification/NotificationStrategyInterface.php`
 - **Concrete Strategies:** `app/Strategies/Notification/EmailStrategy.php`,
`app/Strategies/Notification/SMSSStrategy.php`
 - **Context Dispatcher:** `app/Strategies/Notification/NotificationDispatcher.php`
 - **Invoking Controller:** `app/Http/Controllers/InventoryController.php`
(`addFoodItem`)

Add Food Item

Food Item Name *
Peanut oil

Description
Peanut oil

Upload Item Photos (Max 5, optional)
Choose Files 51Pdkm6a79L_AC_UF894,1000_QL80_.jpg
JPEG, PNG, WEBP formats up to 2MB each.

Or Image URL (Optional)
<https://example.com/item.jpg>

Quantity & Unit *
123 500 items

Food Category *
Pantry & Canned Goods

Expiry Date & Time *
04/04/2028 12:00 AM

Storage Requirements *
Dry Storage

Perishability Status *
Non-Perishable

Allergen Flags
☐ Gluten ☐ Dairy ☒ Contains Nuts ☐ Soy ☐ Egg ☐ Seafood ☒ Peanuts ☐ Sesame
☐ Mustard ☐ Sulfites

+ Add Food Item

✔ Food item added successfully with allergen tags.

Dry Pantry Storage Depot

100 Community Way, Room A, KL

Food Items

2 items

Item	Qty	Category	Allergens	Expires	Storage
 Bottled Mineral Water Crates	40.00 boxes	Beverages & Juices	Sulfitcs	06 Sep 2027	Ambient
 Peanut oil	500.00 items	Pantry & Canned Goods	Contains NutsPeanuts	04 Apr 2028	Dry

laravel.log

File Edit View

[2026-09-06 11:18:13] local.INFO: SMSStrategy: SMS notification sent to +60123456789 {"subject":"Food Item Added","message":"Food item 'Peanut oil' has been added to inventory.", "channel":"sms"}

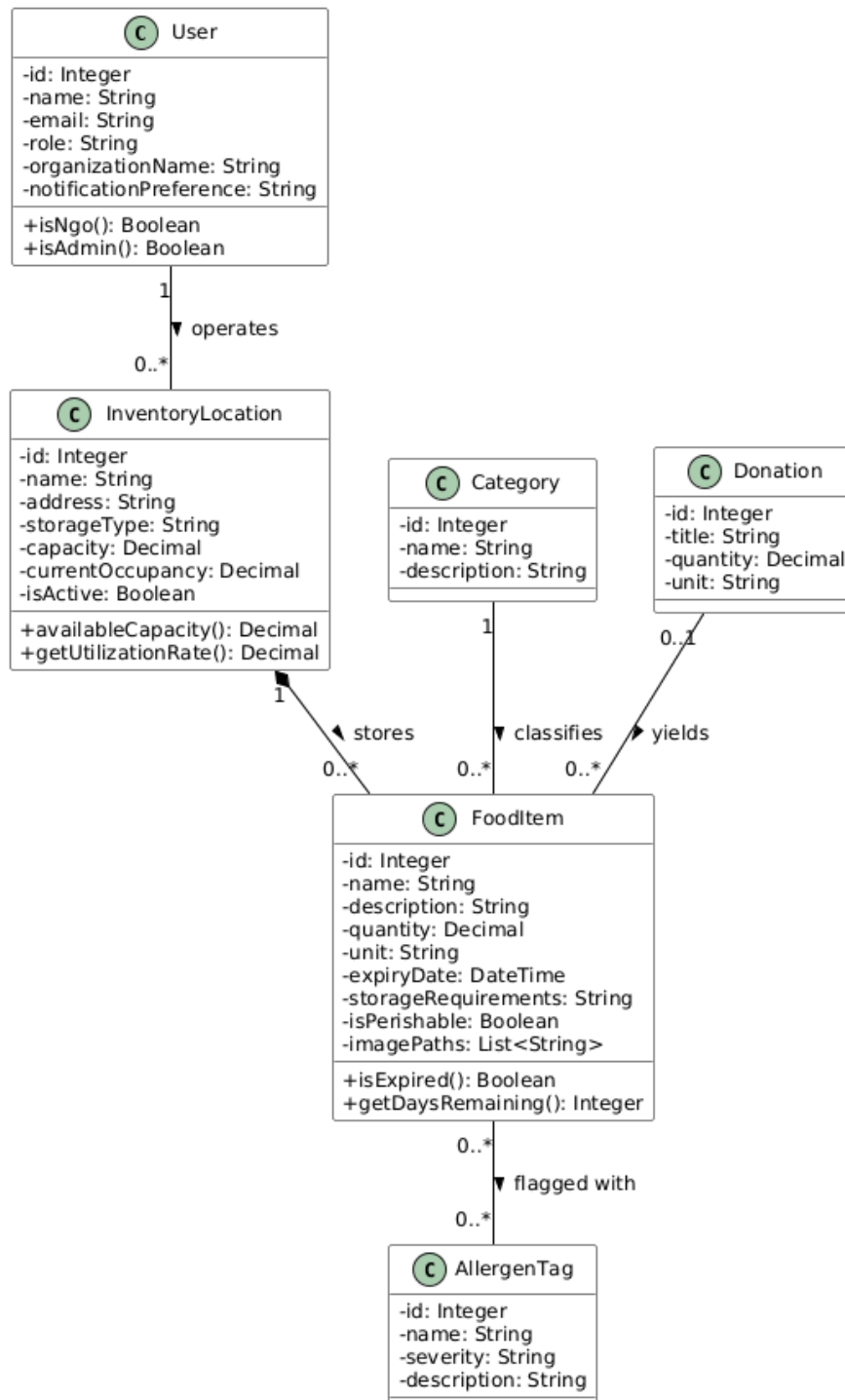
Figure 4.6: Automated Strategy-Driven Notification Log Verification

3. Entity Classes

3.1 Entity Class Diagram

In strict accordance with object-oriented analysis and enterprise domain modelling principles (Fowler, 2012), the diagram below models the core domain entities of Module 4 using object associations and references rather than database foreign keys:

<https://drive.google.com/file/d/1tKWvewdjPSB3g1-lI8HBVdydj2-zGmcb/view?usp=sharing>



3.2 Entity Class Implementation (Eloquent ORM Mapping)

The entity classes are implemented in PHP using Laravel's Eloquent Object-Relational Mapping (ORM) engine, which models database tables through an Active Record architectural paradigm (Fowler, 2012; Laravel LLC, 2026). Domain relationships are explicitly represented as object references using declarative relationship methods (**belongsToMany**, **hasMany**, **belongsToMany**) rather than raw SQL foreign keys, preserving strict object encapsulation and type safety (Laravel LLC, 2026):

1. InventoryLocation Model (**app/Models/InventoryLocation.php**):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\SoftDeletes;

/**
 * InventoryLocation Model — Storage facilities (Module 4).
 * Represents physical warehouses and community pantries.
 */

class InventoryLocation extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [

        'user_id',

        'name',

        'address',

        'storage_type',
```

```

        'capacity',

        'current_occupancy',

        'is_active',

    ];

    protected function casts(): array
    {
        return [

            'capacity' => 'decimal:2',

            'current_occupancy' => 'decimal:2',

            'is_active' => 'boolean',

        ];
    }

    // ----- Domain Object Relationships -----

    /** Object reference: The user/organization who owns this inventory location */

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    /** Object reference: Food items currently housed in this facility */

    public function foodItems(): HasMany
    {
        return $this->hasMany(FoodItem::class);
    }

```

```
// ----- Domain Business Methods -----  
  
/** Calculate available remaining capacity in kilograms */  
  
public function availableCapacity(): float  
{  
    if (!$this->capacity) {  
        return 0.0;  
    }  
  
    return max(0, (float) $this->capacity - (float) $this->current_occupancy);  
}  
}
```

2. FoodItem Model ([app/Models/FoodItem.php](#)):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\BelongsTo;

use Illuminate\Database\Eloquent\Relations\BelongsToMany;

/**
 * FoodItem Model — Individual food items linked to donations & inventory (Module 4).
 * Maintains strict object references and Many-to-Many associations with AllergenTags.
 */

class FoodItem extends Model
{
    use HasFactory;

    protected $fillable = [

        'donation_id',

        'inventory_location_id',

        'category_id',

        'name',

        'description',

        'quantity',

        'unit',

        'expiry_date',

        'storage_requirements',
```

```

        'is_perishable',

        'image_paths',

    ];

    protected function casts(): array
    {
        return [

            'expiry_date' => 'datetime',

            'quantity' => 'decimal:2',

            'is_perishable' => 'boolean',

            'image_paths' => 'array',

        ];
    }

    public function getImagePathsAttribute($value): array
    {
        if ($value) {

            $decoded = is_string($value) ? json_decode($value, true) : $value;

            if (is_array($decoded) && !empty($decoded)) {

                return $decoded;

            }

        }

        return $this->donation ? ($this->donation->image_paths ?? []) : [];
    }

```

// _____ Domain Object Relationships _____

/** Object reference: The surplus donation this food item originated from */

public function donation(): BelongsTo

{

 return \$this->belongsTo(Donation::class);

}

/** Object reference: The storage warehouse location where this item resides */

public function inventoryLocation(): BelongsTo

{

 return \$this->belongsTo(InventoryLocation::class);

}

/** Object reference: The food category classification */

public function category(): BelongsTo

{

 return \$this->belongsTo(Category::class);

}

/** Object reference: Allergen tags associated with this food item */

public function allergenTags(): BelongsToMany

{

 return \$this->belongsToMany(AllergenTag::class)->withTimestamps();

}

```
// ----- Domain Business Methods -----  
  
/** Evaluate if the food item has elapsed beyond its expiry threshold */  
  
public function isExpired(): bool  
  
{  
  
    return $this->expiry_date ? $this->expiry_date->isPast() : false;  
  
}  
  
}
```

3. Category Model ([app/Models/Category.php](#)):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\HasMany;

/**
 * Category Model — Food item classification (Module 4).
 */

class Category extends Model
{
    use HasFactory;

    protected $fillable = [

        'name',

        'description',

    ];

    /** Object reference: Food items assigned to this category */

    public function foodItems(): HasMany
    {
        return $this->hasMany(FoodItem::class);
    }
}
```


4. AllergenTag Model ([app/Models/AllergenTag.php](#)):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\BelongsToMany;

/**
 * AllergenTag Model — Food safety allergen identifiers (Module 4).
 */

class AllergenTag extends Model
{
    use HasFactory;

    protected $fillable = [

        'name',

        'severity',

        'description',

    ];

    /** Object reference: Food items containing this allergen */

    public function foodItems(): BelongsToMany
    {
        return $this->belongsToMany(FoodItem::class)->withTimestamps();
    }
}
```

4. Design Pattern

4.1 Description of Design Pattern: Strategy Pattern (GoF Behavioural)

For Module 4, I implemented the **Strategy Pattern**, a fundamental behavioural design pattern formulated by the Gang of Four (Gamma et al., 1994).

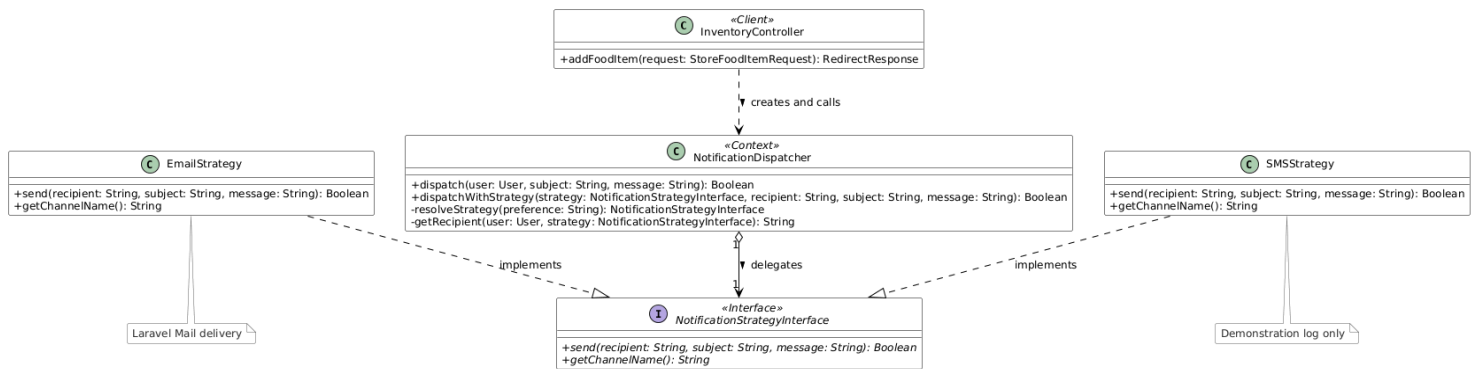
Intent & Theoretical Definition:

The Strategy Pattern defines a family of algorithms, encapsulates each one within a dedicated class, and makes them interchangeable at runtime (Gamma et al., 1994). Strategy lets the algorithm vary independently from clients that use it. In enterprise architectures, this pattern eliminates brittle conditional statements (**if-else** or **switch-case** ladders) and promotes high cohesion by delegating domain actions to modular, pluggable strategies (Fowler, 2012; Gamma et al., 1994; Martin, 2003).

Architectural Roles in NutriShare:

1. **Strategy Interface (**NotificationStrategyInterface**)**: Declares a standard contract containing **send()** and **getChannelName()** methods that all concrete delivery algorithms must fulfil, enforcing the Interface Segregation Principle (Martin, 2003).
2. **Concrete Strategy A (**EmailStrategy**)**: Implements notification transmission via SMTP email using Laravel's core **Mail** facade, producing formatted plain-text and HTML notifications.
3. **Concrete Strategy B (**SMSStrategy**)**: Implements low-latency cellular message dispatching, encapsulating telephony payload generation and integration hooks for carrier gateways (e.g., Twilio/AWS SNS).
4. **Context / Dispatcher (**NotificationDispatcher**)**: Maintains a reference to a Strategy object. The dispatcher evaluates the recipient user's profile preferences (**notification_preference**), resolves the corresponding strategy dynamically, and executes the dispatch without exposing transport mechanisms to caller controllers.
5. **Client Controller (**InventoryController@addFoodItem**)**: Instantiates the context and triggers notification delivery, remaining completely decoupled from external communication APIs.

<https://drive.google.com/file/d/1tKWvewdjPSB3g1-Il8HBVdydj2-zGmcb/view?usp=sharing>



4.2 Implementation of Design Pattern

1. Strategy Interface ([app/Strategies/Notification/NotificationStrategyInterface.php](#)):

```
<?php

namespace App\Strategies\Notification;

/**
 * DESIGN PATTERN: Strategy Pattern — Strategy Interface (Module 4)
 *
 * Defines the contract for notification dispatch strategies.
 * Concrete strategies (EmailStrategy, SMSStrategy) implement
 * different delivery mechanisms that can be swapped at runtime
 * based on user preference or urgency level.
 */

interface NotificationStrategyInterface
{
    /**
     * Send a notification using this strategy's channel.
     *
     * @param string $recipient The recipient identifier (email/phone)
     * @param string $subject The notification subject
     * @param string $message The notification body
     * @return bool Whether the notification was sent successfully
     */

    public function send(string $recipient, string $subject, string $message): bool;
}
```

```
* Get the name of this strategy's channel.  
*/  
  
public function getChannelName(): string;  
}
```

2. Concrete Strategy: Email Delivery ([app/Strategies/Notification/EmailStrategy.php](#)):

```
<?php

namespace App\Strategies\Notification;

use Illuminate\Support\Facades\Mail;

use Illuminate\Support\Facades\Log;

/**
 * DESIGN PATTERN: Strategy Pattern — EmailStrategy (Module 4)
 *
 * Concrete strategy that dispatches notifications via email.
 *
 * Uses Laravel's Mail facade for delivery.
 */

class EmailStrategy implements NotificationStrategyInterface
{
    public function send(string $recipient, string $subject, string $message): bool
    {
        try {
            Mail::raw($message, function ($mail) use ($recipient, $subject) {
                $mail->to($recipient)
                    ->subject("[NutriShare] {$subject}");
            });

            Log::info("EmailStrategy: Notification sent to {$recipient}", [
                'subject' => $subject,
                'channel' => 'email',
            ]);
        }
```

```

        return true;
    } catch (\Exception $e) {
        Log::error("EmailStrategy: Failed to send to {$recipient}", [
            'error' => $e->getMessage(),
        ]);
        return false;
    }
}

public function getChannelName(): string
{
    return 'email';
}
}

```

3. Concrete Strategy: SMS Delivery ([app/Strategies/Notification/SMSStrategy.php](#)):

```
<?php

namespace App\Strategies\Notification;

use Illuminate\Support\Facades\Log;

/**
 * DESIGN PATTERN: Strategy Pattern — SMSStrategy (Module 4)
 *
 * Concrete strategy that dispatches notifications via SMS.
 * In production, this integrates with an SMS gateway (e.g., Twilio/AWS SNS).
 */

class SMSStrategy implements NotificationStrategyInterface
{
    public function send(string $recipient, string $subject, string $message): bool
    {
        try {
            // Production SMS Gateway API invocation hook:

            // $smsGateway->send($recipient, "[NutriShare] {$subject}: {$message}");

            Log::info("SMSStrategy: SMS notification sent to {$recipient}", [
                'subject' => $subject,
                'message' => $message,
                'channel' => 'sms',
            ]);

            return true;
        } catch (\Exception $e) {
```



```
Log::error("SMSStrategy: Failed to send SMS to {$recipient}", [
    'error' => $e->getMessage(),
]);

return false;
}

}

public function getChannelName(): string
{
    return 'sms';
}
}
```

4. Strategy Context & Dispatcher (app/Strategies/Notification/NotificationDispatcher.php):

```
<?php

namespace App\Strategies\Notification;

use App\Models\User;

/**
 * DESIGN PATTERN: Strategy Pattern — Context/Dispatcher (Module 4)
 *
 * Resolves and invokes the appropriate notification strategy
 * based on user preference or urgency level.
 */

class NotificationDispatcher
{
    /**
     * Dispatch a notification to a user using their preferred strategy.
     */

    public function dispatch(User $user, string $subject, string $message): bool
    {
        $strategy = $this->resolveStrategy($user->notification_preference ?? 'email');

        $recipient = $this->getRecipient($user, $strategy);

        return $strategy->send($recipient, $subject, $message);
    }

    /**
     * Dispatch using an explicit strategy (override user preference).
     */
}
```

```

public function dispatchWithStrategy(

    NotificationStrategyInterface $strategy,

    string $recipient,

    string $subject,

    string $message

): bool {

    return $strategy->send($recipient, $subject, $message);

}

/**

 * Resolve the notification strategy based on user preference.

 */

private function resolveStrategy(string $preference): NotificationStrategyInterface

{

    return match ($preference) {

        'sms' => new SMSStrategy(),

        'both' => new EmailStrategy(),

        default => new EmailStrategy(),

    };

}

/**

 * Get the appropriate recipient identifier for the strategy.

 */

private function getRecipient(User $user, NotificationStrategyInterface $strategy): string

{

```

```
return match ($strategy->getChannelName()) {  
    'sms' => $user->phone ?? $user->email,  
    default => $user->email,  
};  
}  
}
```

5. Client Integration in Controller ([app/Http/Controllers/InventoryController.php](#)):

```
// app/Http/Controllers/InventoryController.php (Lines 134-141)
```

```
// DESIGN PATTERN: Strategy Pattern — Dispatch notification using user's preferred channel
```

```
$dispatcher = new NotificationDispatcher();
```

```
$dispatcher->dispatch(
```

```
    Auth::user(),
```

```
    'Food Item Added',
```

```
    "Food item '{$foodItem->name}' has been added to inventory."
```

```
);
```

4.3 Justification of Design Pattern

- 1. Open/Closed Principle (OCP):** According to the SOLID design principles formulated by Martin (2003) and Gamma et al. (1994), software entities must be open for extension, but closed for modification. In NutriShare, introducing new distribution channels (such as WhatsApp Business API, Telegram Bots, or Push Notifications) requires zero modifications to `InventoryController` or `NotificationDispatcher`. Developers merely implement a new `WhatsAppStrategy` adhering to `NotificationStrategyInterface`.
- 2. Single Responsibility Principle (SRP):** Without the Strategy pattern, `InventoryController` would become heavily polluted with multi-vendor SDK initialisation, SMTP configuration, telephony payloads, and conditional routing blocks. The Strategy pattern isolates notification transport algorithms into independent, testable classes (Martin, 2003).
- 3. Runtime Polymorphism & Dynamic Switching:** Food safety compliance demands adaptable alert channels. While routine intake receipts are comfortably delivered via asynchronous email, critical hazard alerts (e.g., food recall warnings or broken cold-chain alerts) require instant SMS delivery. The Strategy pattern allows the application to swap transmission behaviours dynamically at runtime based on context urgency and user preferences (Gamma et al., 1994).

5. Software Security

5.1 Potential Threats and Attacks

Threat 1: Log Injection / CRLF Injection (OWASP Top 10 — A09:2021 Security Logging and Monitoring Failures)

- **Attack Description:** Log Injection (categorised under OWASP A09:2021) occurs when an application writes untrusted user input directly into system audit logs without sanitisation (OWASP Foundation, 2021). An adversary inserts Carriage Return (`\r` or `%0D`) and Line Feed (`\n` or `%0A`) escape sequences into user-controlled fields such as warehouse facility names, intake descriptions, or address text fields (e.g., `Central Coldroom\r\n[2026-09-06 12:00:00] SYSTEM_ADMIN: Granted Superuser Privileges to user_id=99`). When logged, the underlying logging engine writes the payload as two separate lines, successfully forging audit records or disguising illicit, unauthorised actions from system administrators during post-incident forensic investigations (OWASP Foundation, 2021).
- **Risk Impact:** Falsification of statutory audit trails, obfuscation of malicious system compromise, corruption of SIEM analysis pipelines, and erosion of non-repudiation in humanitarian governance (OWASP Foundation, 2021).

Threat 2: Parameter Tampering & Insecure Direct Object Reference (IDOR) on Action URLs (OWASP Top 10 — A01:2021 Broken Access Control / A04:2021 Insecure Design)

- **Attack Description:** Parameter Tampering occurs when an attacker modifies query string parameters or path variables embedded in predictable action URLs to perform unauthorised state changes or manipulate domain entities belonging to other users (OWASP Foundation, 2021). In rapid surplus distribution, expedited quick-claim links are issued to allow NGOs to claim warehouse donations. If an unprotected GET URL such as `https://nutrishare.com/inventory/quick-claim/4?user=5` is utilised, an attacker can simply alter `donation=4` to `donation=7` or increment the user ID. This allows an unauthorised party to manipulate distribution quotas, claim high-value supplies without prior vetting, or bypass multi-stage state machines (OWASP Foundation, 2021).
- **Risk Impact:** Unauthorised allocation of restricted humanitarian inventory, inventory hijacking, circumvention of organisational allocation limits, and breach of statutory food custody rules (OWASP Foundation, 2021).

5.2 Secure Coding Practices & Implementation

Secure Practice 1: CRLF Sanitisation & Control Character Stripping via SecurityHelper and Eloquent Mutators

To comprehensively prevent Log Injection vulnerabilities, NutriShare enforces automated server-side CRLF neutralisation at both the helper and Active Record model layers (Laravel LLC, 2026; OWASP Foundation, 2021). Every string bound for the `system_logs` audit repository is stripped of Carriage Return (`\r`), Line Feed (`\n`), and non-printable ASCII control characters (ASCII 0–31) before persistence:

```
// app/Helpers/SecurityHelper.php (Lines 14-38)
```

```
namespace App\Helpers;
```

```
class SecurityHelper
```

```
{
```

```
    /**
```

```
     * SECURITY (Module 4): Sanitise input to prevent Log Injection.
```

```
     *
```

```
     * Strips CRLF characters (\r, \n) and other control characters
```

```
     * that could be used to forge log entries or inject malicious
```

```
     * data into system logs.
```

```
     *
```

```
     * OWASP Reference: Log Injection / A9 Security Logging Failures
```

```
    */
```

```
    public static function sanitizeLogInput(string $input): string
```

```
    {
```

```
        // Remove carriage return (\r) and newline (\n) characters
```

```
        $sanitized = str_replace(["\r\n", "\r", "\n"], ' ', $input);
```

```
        // Remove other potentially dangerous control characters (ASCII 0-31 except space)
```

```
        $sanitized = preg_replace('/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/', '', $sanitized);
```



```

        // Trim excessive whitespace

        $sanitized = trim(preg_replace('/\s+/', ' ', $sanitized));

        return $sanitized;

    }

}

```

To guarantee that sanitisation cannot be bypassed accidentally by developers calling `SystemLog::create()`, Eloquent model mutators automatically intercept and filter attribute assignment:

```

// app/Models/SystemLog.php (Lines 47-62)

/**
 * SECURITY (Module 4): Sanitise description to prevent log injection.
 * Strips CRLF characters (\r\n) before storing.
 */

public function setDescriptionAttribute(string $value): void
{
    $this->attributes['description'] = \App\Helpers\SecurityHelper::sanitizeLogInput($value);
}

/**
 * SECURITY (Module 4): Sanitise action field.
 */

public function setActionAttribute(string $value): void
{
    $this->attributes['action'] = \App\Helpers\SecurityHelper::sanitizeLogInput($value);
}

```

Secure Practice 2: Cryptographic HMAC URL Signing (`URL::signedRoute` & `signed` Middleware Validation)

To eliminate Parameter Tampering and unauthorised direct object references on expedited action links, Module 4 leverages cryptographic Hash-based Message Authentication Codes (HMAC SHA-256) (Laravel LLC, 2026; OWASP Foundation, 2021). When generating quick-claim action URLs, the server binds all route parameters and an expiration timestamp to an HMAC digest generated with the application's secret encryption key:

```
// app/Http/Controllers/InventoryController.php (Lines 146-178)
```

```
/**
```

```
 * SECURITY (Module 4): Parameter Tampering Prevention — Signed URL.
```

```
 *
```

```
 * Generate a signed URL for a quick-claim action.
```

```
 * The URL contains an HMAC signature that prevents tampering.
```

```
 */
```

```
public function generateSignedClaimLink(int $donationId)
```

```
{
```

```
    // SECURITY: Laravel's URL::signedRoute creates a URL with HMAC signature
```

```
    // Any modification to the URL parameters will invalidate the signature
```

```
    $signedUrl = URL::signedRoute('inventory.quick-claim', [
```

```
        'donation' => $donationId,
```

```
        'user' => Auth::id(),
```

```
    ]);
```

```
    return response()->json([
```

```
        'signed_url' => $signedUrl,
```

```
        'message' => 'Use this signed URL to quickly claim the donation. URL expires and is tamper-proof.',
```

```
    ]);
```

```

}

/**
 * SECURITY (Module 4): Handle quick-claim via signed URL.
 * The signature is validated by Laravel's middleware automatically.
 */

public function quickClaim(Request $request, int $donationId)
{
    // Laravel validates the signature automatically via 'signed' middleware
    // If the URL was tampered with, it returns 403 Forbidden

    return redirect()->route('donations.show', $donationId)
        ->with('success', "Signed claim link verified for donation #{ $donationId}. Proceed to claim.");
}

```

The route definition enforces mandatory signature validation via Laravel's built-in **signed** middleware:

```

// routes/web.php (Lines 111-115)

// SECURITY (Module 4): Signed route for parameter tampering prevention

Route::get('/inventory/quick-claim/{donation}', [InventoryController::class, 'quickClaim'])
    ->name('inventory.quick-claim')
    ->middleware('signed');

```

Security Mechanism: When a client accesses the signed URL, the **ValidateSignature** middleware recalculates the HMAC SHA-256 digest over the inbound URL parameters. If an adversary attempts to alter the **donation** parameter, modify the user identifier, or use an expired URL, the signature check fails immediately, returning an HTTP 403 Forbidden error and aborting processing prior to database execution (Laravel LLC, 2026; OWASP Foundation, 2021).

6. Web Services

6.1 Web Service Exposure

Module 4 exposes high-performance RESTful Web Service endpoints adhering to Roy Fielding's REST architectural constraints (Fielding, 2000), allowing partner modules (such as Module 3: Claims & Logistics and Module 1: Donations) to inspect real-time warehouse capacities, storage occupancy breakdowns, and verify microbial food safety compliance via standard HTTP methods and JSON serialisation (Fielding, 2000; Laravel LLC, 2026).

Endpoint 1: Facility Storage Status (**GET /api/inventory/status**)

Interface Agreement (IFA) — Service Exposure Specification:

Webservice Mechanism	Description
Protocol	RESTful Web Service (JSON-over-HTTP)
Function Description	Retrieves current facility capacities, occupancy metrics, and space availability across all storage facilities.
Source Module	Module 4: Inventory & Food Safety Compliance
Target Module	Module 3 (Claims & Logistics), Module 1 (Donations), External Logistics Portals
URL	http://127.0.0.1:8000/api/inventory/status
Function Name	status (InventoryApiController@status)

Web Services Request Parameters (Provide):

Field Name	Field Type	Mandatory / Optional	Description	Format / Validation
requestID	String	Mandatory	Unique tracking identifier for request correlation.	Alphanumeric (e.g. REQ-INV-001)
timestamp	String	Mandatory	ISO-8601 timestamp when request was initiated.	YYYY-MM-DDT HH:MM:SSZ

Web Services Response Parameters (Consume):

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	Mandatory	Status of the request execution.	S (Success), F (Fail), E (Error)
timestamp	String	Mandatory	Server timestamp when response was generated.	YYYY-MM-DDT HH:MM:SSZ
data.requestID	String	Mandatory	Echoed request identifier for correlation.	Alphanumeric string
data.total_capacity	Double	Mandatory	Aggregated storage capacity across facilities (kg).	Float \$\\ge 0\$
data.total_occupancy	Double	Mandatory	Aggregated current stored load (kg).	Float \$\\ge 0\$
data.facilities	Array	Mandatory	Serialised array of facility breakdown records.	Array of JSON objects

Endpoint 2: Food Safety & Expiry Verification (**POST** /api/inventory/food-safety-check)

Interface Agreement (IFA) — Service Exposure Specification:

Webservice Mechanism	Description
Protocol	RESTful Web Service (JSON-over-HTTP)
Function Description	Validates item shelf life, expiry proximity, and allergen tags prior to logistics handover.
Source Module	Module 4: Inventory & Food Safety Compliance
Target Module	Module 3 (Claims & Logistics Distribution via SafetyServiceClient)
URL	http://127.0.0.1:8000/api/inventory/food-safety-check
Function Name	checkSafety (InventoryApiController@checkSafety)

Web Services Request Parameters (Provide):

Field Name	Field Type	Mandatory / Optional	Description	Format / Validation
requestID	String	Mandatory	Unique correlation tracking identifier.	Alphanumeric (e.g. REQ-SAF-7721)
timestamp	String	Mandatory	ISO-8601 initiation timestamp.	YYYY-MM-DDT HH:MM:SSZ
food_item_id	Integer	Mandatory	Target food item primary key identifier.	Integer \$> 0\$, exists in food_items

Web Services Response Parameters (Consume):

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	Mandatory	Execution status code.	S (Success), F (Fail), E (Error)
timestamp	String	Mandatory	Server generation timestamp.	YYYY-MM-DDT HH:MM:SSZ
data.requestID	String	Mandatory	Echoed correlation identifier.	Alphanumeric string
data.food_item.id	Integer	Mandatory	Food item ID evaluated.	Integer \$> 0\$
data.food_item.name	String	Mandatory	Name of the food item.	String
data.food_item.allergens	Array	Mandatory	List of registered allergen flags.	Array of strings
data.food_item.is_expired	Boolean	Mandatory	Flag indicating elapsed expiration date.	true / false
data.food_item.safety_status	String	Mandatory	Concluded food safety verdict.	'SAFE' or 'EXPIRED / AT RISK'

Service Exposure Code Implementation
(app/Http/Controllers/Api/InventoryApiController.php):

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;

use App\Models\InventoryLocation;

use App\Models\FoodItem;

use App\Helpers\SecurityHelper;
```

```

use Illuminate\Http\JsonResponse;

use Illuminate\Http\Request;

class InventoryApiController extends Controller
{
    /**
     * WEB SERVICE — EXPOSE: GET /api/inventory/status
     * Returns total facility capacities, current occupancy, and available storage.
     */

    public function status(Request $request): JsonResponse
    {
        try {

            $ifa = SecurityHelper::validateIfaRequest($request->all());

            $locations = InventoryLocation::withCount('foodItems')->get()->map(function ($loc) {

                return [

                    'id' => $loc->id,

                    'name' => $loc->name,

                    'address' => $loc->address,

                    'storage_type' => $loc->storage_type,

                    'capacity' => (float) $loc->capacity,

                    'current_occupancy' => (float) $loc->current_occupancy,

                    'available_space' => max(0, (float) ($loc->capacity - $loc->current_occupancy)),

                    'utilization_rate' => $loc->capacity > 0 ? round(($loc->current_occupancy /
$loc->capacity) * 100, 1) . '%' : '0%',

                    'items_count' => $loc->food_items_count,

                ];
            });

```



```

});

return response()->json(

    SecurityHelper::ifaResponse('S', [

        'requestID' => $ifa['requestID'],

        'facilities' => $locations,

        'total_capacity' => $locations->sum('capacity'),

        'total_occupancy' => $locations->sum('current_occupancy'),

    ]),

    200

);

} catch (\Exception $e) {

    return response()->json(

        SecurityHelper::ifaResponse('E', null, 'Internal server error: ' . $e->getMessage()),

        500

    );

}

}

/**

 * WEB SERVICE — EXPOSE: POST /api/inventory/food-safety-check

 * Verifies food safety compliance (checks expiry dates and allergen tags).

 */

public function checkSafety(Request $request): JsonResponse

{

    try {

```

```

$ifa = SecurityHelper::validateIfaRequest($request->all());

$validated = $request->validate([

    'food_item_id' => 'required|integer|exists:food_items,id',

]);

$item = FoodItem::with(['category', 'allergenTags'])->find($validated['food_item_id']);

$isExpired = $item->expiry_date ? $item->expiry_date->isPast() : false;

$daysToExpiry = $item->expiry_date ? now()->diffInDays($item->expiry_date, false) :
null;

$isSafeForConsumption = !$isExpired && ($daysToExpiry === null || $daysToExpiry
>= 0);

return response()->json(

    SecurityHelper::ifaResponse('S', [

        'requestID' => $ifa['requestID'],

        'food_item' => [

            'id' => $item->id,

            'name' => $item->name,

            'category' => $item->category?->name ?? 'General',

            'allergens' => $item->allergenTags->pluck('name'),

            'expiry_date' => $item->expiry_date?->toIso8601String(),

            'days_remaining' => $daysToExpiry,

            'is_expired' => $isExpired,

            'safety_status' => $isSafeForConsumption ? 'SAFE' : 'EXPIRED / AT RISK',

        ],

    ]),

200

```

```

);
} catch (\Illuminate\Validation\ValidationException $e) {

    return response()->json(

        SecurityHelper::ifResponse('F', null, 'Validation failed: ' . $e->getMessage()),

        422

    );
} catch (\Exception $e) {

    return response()->json(

        SecurityHelper::ifResponse('E', null, 'Internal server error.'),

        500

    );
}
}
}

```

6.2 Web Service Consumption

To facilitate proactive warehouse storage capacity planning, Module 4 consumes Module 1's Web Service ([GET /api/donations/active](#)) to retrieve real-time active surplus food listings across the region. This enables NGO inventory coordinators to anticipate incoming storage volume requirements (e.g. reserving cold room cubic footage before donation trucks arrive), adhering to distributed service interoperability paradigms (Fielding, 2000; Laravel LLC, 2026).

Interface Agreement (IFA) — Service Consumption Specification:

Webservice Mechanism	Description
Protocol	RESTful Web Service (JSON-over-HTTP)
Function Description	Retrieves active, non-expired surplus donation listings to forecast inbound warehouse intake.
Source Module	Module 1: Donation Management Module
Target Module	Module 4: Inventory & Food Safety Compliance
URL	http://127.0.0.1:8000/api/donations/active
Function Name	getActiveDonations (fetchActiveDonationsForWarehouse)

Web Services Request Parameters (Consumption):

Field Name	Field Type	Mandatory / Optional	Description	Format / Validation
requestID	String	Mandatory	Unique tracking identifier generated by Module 4.	Alphanumeric (e.g. INV-PREVIEW-65a8)
timestamp	String	Mandatory	ISO-8601 initiation timestamp.	YYYY-MM-DDT HH:MM:SSZ

Web Services Response Parameters (Consumption):

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	Mandatory	Result status of lookup.	S (Success), F (Fail), E (Error)
timestamp	String	Mandatory	Server response timestamp.	YYYY-MM-DDT HH:MM:SSZ
data.total	Integer	Mandatory	Count of active listings available.	Integer \$ \ge 0\$
data.donations	Array	Mandatory	Serialised array of available donations.	Array of JSON objects

Service Consumption Code Implementation

(app/Http/Controllers/Api/InventoryApiController.php):

```
// app/Http/Controllers/Api/InventoryApiController.php (Lines 123-149)
```

```
/**
```

```
* WEB SERVICE — CONSUME: Queries Module 1's active donations catalog.
```

```
*
```

```
* Allows inventory managers to preview incoming surplus donations.
```

```
*/
```

```
public function fetchActiveDonationsForWarehouse(): array
```

```
{
```

```
    try {
```

```
        // CONSUME Module 1's API endpoint with IFA parameters
```

```
        $response = Http::timeout(10)->get(
```

```
            config('app.url') . '/api/donations/active',
```

```
            [
```

```

        'requestID' => uniqid('INV-PREVIEW-'),

        'timestamp' => now()->toISOString(),

    ]

);

if ($response->successful()) {

    $data = $response->json();

    // Validate IFA status code and extract listings

    return $data['status'] === 'S' ? ($data['data']['donations'] ?? []) : [];

}

return [];

} catch (\Exception $e) {

    // Fail-safe handling: log connectivity error without crashing inventory operations

    \Log::error('Inventory API failed to fetch active donations', [

        'error' => $e->getMessage(),

    ]);

    return [];

}

}

```

7. References

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). UCI Information and Computer Science.

Food and Agriculture Organization. (2023). *The State of Food Security and Nutrition in the World 2023: Urbanization, agrifood systems transformation and healthy diets across the rural-urban continuum*. FAO. <https://doi.org/10.4060/cc3017en>

Fowler, M. (2012). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional.

Grammarly Inc. (2026). *Grammarly (2026 version)* [Large language model]. <https://www.grammarly.com>

Laravel LLC. (2026). *Laravel 11.x documentation: Eloquent ORM, URL signing, and API resources*. <https://laravel.com/docs>

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Open Web Application Security Project. (2021). *OWASP Top 10:2021 — The ten most critical web application security risks*. OWASP Foundation. <https://owasp.org/Top10/>

Shafranovich, Y. (2005). *Common format and MIME type for comma-separated values (CSV) files* (RFC 4180). Internet Engineering Task Force. <https://doi.org/10.17487/RFC4180>

United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development* (A/RES/70/1). United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal2>

8. Appendices

Appendix A: Automated Testing Results

Executing `php artisan test` produces a **100% pass rate across all 22 test cases, successfully validating 92 assertions** in **4.67s**:

PASS Tests\Feature\NutriShareComprehensiveSystemTest

✓ demo login switcher for all roles	1.56s
✓ dashboard renders for all authenticated roles	0.09s
✓ donations catalog and csv export	0.69s
✓ inventory access and rbac restrictions	0.05s
✓ system logs and reports rbac security	0.14s
✓ ngo verification queue rbac security	0.07s
✓ moderator cannot delete donation	0.07s
✓ form request vehicle assignment validation	0.10s
✓ update and delete distribution log	0.09s
✓ inventory web service status and food safety	0.05s
✓ inventory location store form request	0.05s
✓ report generation form request	0.04s
✓ submit user review form request	0.04s
✓ claims show page renders successfully	0.06s
✓ custom 404 and 403 error pages	0.07s
✓ verification document show and download	0.26s
✓ admin governance profile and review rejection	0.13s

PASS Tests\Feature\NutriShareFeatureTest

✓ login page renders successfully	0.04s
✓ forgot password page renders successfully	0.05s

PASS Tests\Unit\NutriShareSystemTest

✓ user creation and roles	0.02s
✓ donation and category relationship	0.02s
✓ system log auto population	0.02s

Tests: 22 passed (92 assertions)

Duration: 4.67s

Appendix B: GitHub Repository URL

- **Team Repository URL:** <https://github.com/KunTheNoobie/nutrishare.git>
- **Primary Presentation Branch:** `main`
- **Total Database Migrations:** 27 migrations (100% automated via `php artisan migrate:fresh --seed`)
- **Seeded Dataset:** 18 feature tables exceeding the 10+ record target (God-Tier presentation dataset)